

Сувеніри

Амару купує сувеніри в іноземному магазині. Є N **типів** сувенірів. У магазині є нескінченна кількість сувенірів кожного типу.

Кожен тип сувеніра має фіксовану ціну. А саме, сувенір типу i ($0 \leq i < N$) має ціну $P[i]$ монет, де $P[i]$ - додатне число.

Амару відомо, що типи сувенірів відсортовано за спаданням ціни, і що ціни на сувеніри різні. Зокрема, $P[0] > P[1] > \dots > P[N-1] > 0$. Крім того, він зміг дізнатися вартість $P[0]$. На жаль, Амару не має іншої інформації про ціни на сувеніри.

Щоб купити сувеніри, Амару здійснить низку транзакцій з продавцем.

Кожна транзакція складається з наступних кроків:

1. Амару передає деяку (додатну) кількість монет продавцю.
2. Продавець складає ці монети в купу на столі в задній кімнаті, де Амару не зможе їх побачити.
3. Продавець розглядає кожен тип сувенірів $0, 1, \dots, N-1$ в такому порядку, один за одним. Кожен тип розглядається **рівно один раз** за транзакцію.
 - При розгляді сувеніра типу i , якщо поточна кількість монет у стопці становить не менше $P[i]$, то
 - продавець прибирає зі стопки $P[i]$ монет, і
 - продавець кладе на стіл один сувенір типу i .
4. Продавець віддає Амару всі монети, що залишилися в купі, та всі сувеніри на столі.

Зверніть увагу, що перед початком транзакції на столі немає монет або сувенірів.

Ваше завдання полягає в тому, щоб допомогти Амару виконати деяку кількість транзакцій так, щоб:

- в кожній транзакції він купує **принаймні один** сувенір, і
- загалом він купує **рівно i** сувенірів типу i , для кожного i такого, що $0 \leq i < N$. Зверніть увагу, що це означає, що Амару не має купувати сувеніри типу 0.

Амару не потрібно мінімізувати кількість транзакцій і має необмежений запас монет.

Деталі реалізації

Вам слід реалізувати наступну функцію.

```
void buy_souvenirs(int N, long long P0)
```

- N : кількість типів сувенірів.
- $P0$: значення $P[0]$.
- Ця функція викликається рівно один раз для кожного тесту.

Вищевказана функція може здійснювати виклики до наступної функції, щоб допомогти Амару виконати транзакцію:

```
std::pair<std::vector<int>, long long> transaction(long long M)
```

- M кількість монет, які Амару передав продавцю.
- Функція повертає пару. Першим елементом пари є масив L , що містить типи сувенірів, які було куплено (у порядку зростання). Другий елемент - це ціле число R , кількість монет, які повертаються Амару після транзакції.
- Потрібно, щоб $P[0] > M \geq P[N - 1]$. Умова $P[0] > M$ гарантує, що Амару не купить жодного сувеніра типу 0, а $M \geq P[N - 1]$ гарантує, що Амару купить хоча б один сувенір. Якщо ці умови не виконуються, ваш розв'язок отримає вердикт `Output isn't correct: Invalid argument`. Зверніть увагу, що на відміну від $P[0]$, значення $P[N - 1]$ у вхідних даних не задається.
- Функція може бути викликана не більше 5000 разів у кожному тесті.

Поведінка градера **не адаптивна**. Це означає, що послідовність цін P фіксується до виклику `buy_souvenirs`.

Обмеження

- $2 \leq N \leq 100$
- $1 \leq P[i] \leq 10^{15}$ для всіх i таких, що $0 \leq i < N$.
- $P[i] > P[i + 1]$ для всіх i таких, що $0 \leq i < N - 1$.

Підзадачі

Підзадача	Бали	Додаткові обмеження
1	4	$N = 2$
2	3	$P[i] = N - i$ для всіх i таких, що $0 \leq i < N$.
3	14	$P[i] \leq P[i + 1] + 2$ для всіх i таких, що $0 \leq i < N - 1$.
4	18	$N = 3$
5	28	$P[i + 1] + P[i + 2] \leq P[i]$ для всіх i таких, що $0 \leq i < N - 2$. $P[i] \leq 2 \cdot P[i + 1]$ для всіх i таких, що $0 \leq i < N - 1$.
6	33	Без додаткових обмежень.

Приклад

Розглянемо наступний виклик.

```
buy_souvenirs(3, 4)
```

Існує $N = 3$ видів сувенірів та $P[0] = 4$. Зверніть увагу, що існує лише три можливі послідовності цін P : $[4, 3, 2]$, $[4, 3, 1]$, та $[4, 2, 1]$.

Припустимо, що `buy_souvenirs` викликає `transaction(2)`. Припустимо, що виклик повертає $([2], 1)$, що означає, що Амару купив один сувенір типу 2 і продавець повернув йому одну монету. Зауважте, що це дозволяє нам зробити висновок, що $P = [4, 3, 1]$, оскільки:

- Для $P = [4, 3, 2]$, `transaction(2)` повернула б $([2], 0)$.
- Для $P = [4, 2, 1]$, `transaction(2)` повернула б $([1], 0)$.

Тоді `buy_souvenirs` може викликати `transaction(3)`, яка повертає $([1], 0)$, що означає, що Амару купив один сувенір типу 1 і продавець повернув йому 0 монет. Поки що, загалом, він купив один сувенір типу 1 і один сувенір типу 2.

Нарешті, `buy_souvenirs` може викликати `transaction(1)`, яка повертає $([2], 0)$, що означає, що Амару купив один сувенір типу 2. Зауважте, що тут також можна було б використати `transaction(2)`. На даний момент, загалом Амару має один сувенір типу 1 і два сувеніри типу 2, як і потрібно.

Приклад градера

Формат вхідних даних:

N

$P[0] \quad P[1] \quad \dots \quad P[N-1]$

Формат вихідних даних:

$Q[0] \quad Q[1] \quad \dots \quad Q[N-1]$

Тут $Q[i]$ - це кількість сувенірів типу i , куплених загалом для кожного i такого, що $0 \leq i < N$.